Lab 2 - Cloud Data, Stat 214, Spring 2025

March 20, 2026

1 Introduction

Cloud detection in polar regions is a fundamental challenge in remote sensing. While standard cloud detection algorithms rely on brightness thresholds, since clouds appear bright and surfaces appear dark, this is not necessarily true in the Artic, where sea ice and snow are equally bright and cold as clouds. Cloud classification is particularly relevant for climate modeling.

This report analyzes satellite imagery from the Multi-angle Imaging SpectroRadiometer (MISR), a sensor aboard NASA's Terra satellite that captures the same scene simultaneously from 9 camera angles. The key insight is that clouds and ice scatter light differently depending on viewing angle: ice scatters roughly uniformly across angles, while clouds scatter strongly in the forward direction. This angular variation is the physical signal that enables classification where simple brightness-based approaches fail.

Our analysis proceeds in four stages. First, we perform exploratory data analysis to understand the structure of the data and the discriminative power of raw and engineered features. Second, we engineer additional features using an autoencoder to capture local spatial texture. Third, we train and evaluate classifiers for pixel-level cloud detection. Finally, we assess model stability through perturbation analysis and apply the classifier to unlabeled images as a qualitative sanity check.

1.1 Data

The dataset consists of three labeled MISR images of the same Arctic geographic region (path 26), collected during the 2002 Arctic daylight season. Each image corresponds to a distinct satellite overpass:

Orbit	**Approximate Date**	**Season**
O012791	Late April / Early May	Spring
O013257	Late May / Early June	Spring
O013490	Mid June	Early Summer

Table 1: Orbit Dates and Corresponding Seasons

Each pixel covers a $275\,\mathrm{m} \times 275\,\mathrm{m}$ patch and is recorded at the red band across five camera angles: DF (70.5°), CF (60°), BF (45.6°), AF (26.1°), and AN (nadir, 0°). The dataset contains approximately 345,000 total pixels across the three images, of which $\sim$ 207,000 carry expert labels (+1 cloud, −1 not cloud), with the remainder unlabeled (0) at ambiguous cloud boundaries.

In additional to the raw angular radiances features, each pixel carries three engineered features derived from the angular radiances, designed to capture the physics of polar cloud-surface classification:

- NDAI (Normalized Difference Angular Index): ratio comparing the most extreme forward camera (DF) to nadir (AN). Large positive values indicate strong forward scattering, a cloud signature.
- SD: standard deviation of nadir radiance in a local neighborhood. Clouds produce higher texture variation than smooth ice.
- CORR: average cross-angle correlation between camera pairs. High correlation indicates the same surface is seen from each angle (ice); low correlation indicates a cloud.

While the raw radiances are included as features, they provide limited discriminative power on their own: in the Arctic, both clouds and snow/ice surfaces are highly reflective, so brightness alone cannot reliably distinguish between them. The engineered features (NDAI, SD, CORR, etc.) are specifically designed to

exploit angular variation rather than absolute brightness, and are the primary drivers of class separation.

2 EDA

2.1 Spatial Structure of the Labeled Images

Figure 1 shows the expert label maps for all three orbits. A few things stand out immediately. Clouds form large, spatially coherent regions rather than isolated pixels, neighboring pixels tend to share the same label, which matters for how we evaluate model performance. Unlabeled pixels cluster almost entirely at cloud edges, exactly where expert labelers were uncertain, so the ambiguity in the dataset has a clear geographic logic rather than being random noise. Finally, cloud coverage and arrangement shift meaningfully across the three orbits, reflecting the seasonal transition from spring to early summer over the same Arctic region.

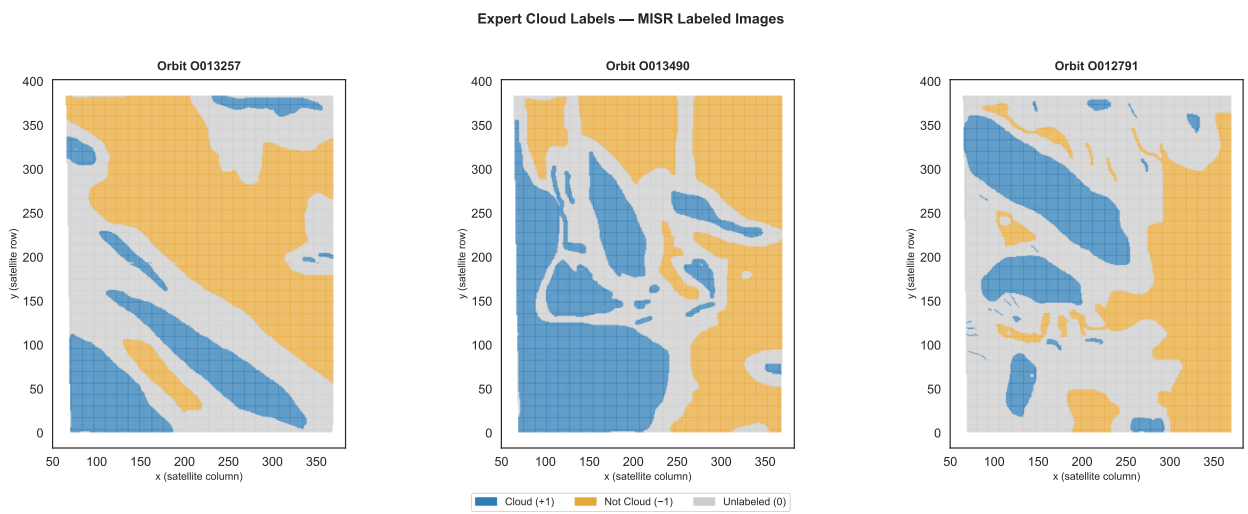

Figure 1: Expert label maps for the three MISR orbits (O012791, O013257, O013490). Blue pixels are labeled cloud (+1), yellow are not-cloud (−1), and gray are unlabeled (0). Clouds form spatially coherent regions; unlabeled pixels concentrate at cloud-surface boundaries.

2.2 Data Quality

Before modeling, we verified the integrity of the dataset along several dimensions. There are no missing values, no physically invalid radiances (zero or negative), no correlation values outside $[-1, 1]$, and no duplicate pixel coordinates within any image. The data requires no imputation or structural repair.

We identified outliers as pixels exceeding three standard deviations from the mean in at least one feature, amounting to 6.3% of all pixels. Rather than being randomly scattered, these pixels cluster geographically in the lower-left region of the image grid, corresponding to the Greenland coastline, a region flagged by Shi et al. (2008) as problematic due to poor cross-angle terrain registration at sharp elevation changes. This geographic coherence indicates the outliers reflect genuine extreme surface conditions rather than sensor artifacts, so we retain all pixels, while noting that model performance may be lower in this coastal region.

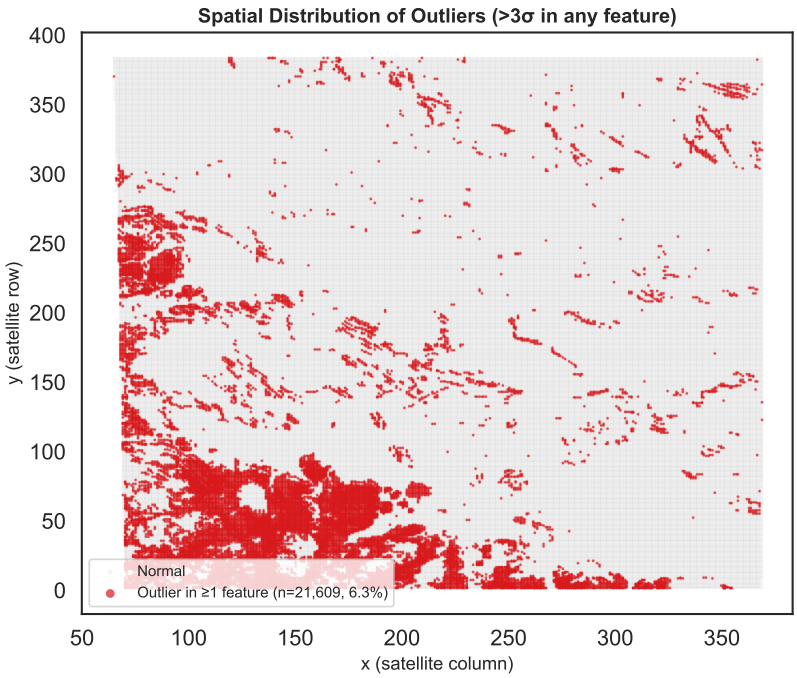

Figure 2: Spatial distribution of outlier pixels (red), defined as exceeding 3σ in at least one of the eight features. Outliers cluster in the lower-left region of the image grid, consistent with the Greenland coastal terrain where poor cross-angle registration produces extreme feature values.

2.3 Feature Analysis: Radiances and Engineered Features

A. Correlations Among Raw Radiance Angles

Since all five cameras photograph the same scene simultaneously, their radiances are expected to be correlated. Figure 3 confirms this: adjacent angles (e.g., AF and AN) are nearly perfectly correlated ($r = 0.97$), while the most geometrically separated pair (DF and AN) shows the weakest relationship ($r = 0.60$), still strongly positive, but with meaningful variation. This makes a lot of physical sense: nearby angles view nearly the same surface geometry, while more extreme angles begin to capture different scattering behavior.

The high multicollinearity among raw radiances has practical consequences for modeling. In linear models, highly correlated features produce unstable coefficients that are difficult to interpret. More importantly, it means the five raw radiances do not carry five independent signals, much of their information overlaps. The engineered features are designed to extract the residual, physically meaningful variation that the correlations leave behind.

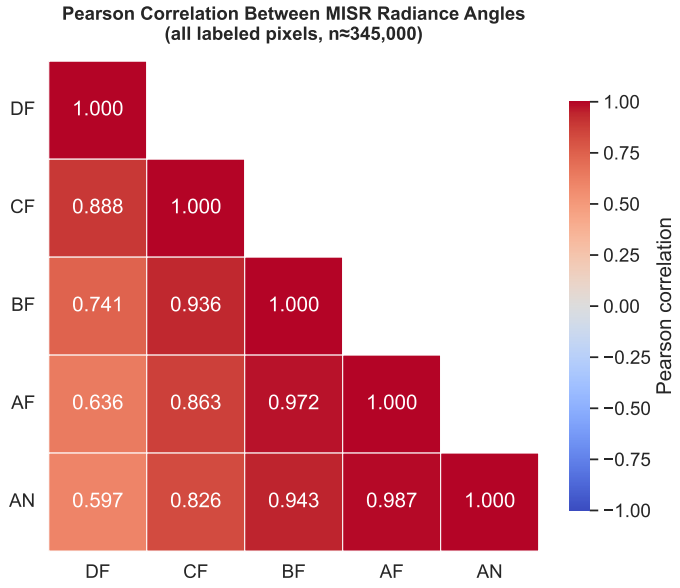

Figure 3: Pearson correlation matrix of the five MISR radiance angles across all labeled pixels ($n \approx 345{,}000$). Correlations decrease monotonically with angular separation, from $r = 0.97$ (AF–AN) to $r = 0.60$ (DF–AN).

B. Class Separation in Raw Radiances vs. Engineered Features

Figure 4 shows the distribution of all eight features broken down by class. For raw radiances (DF through AN), the cloud and not-cloud distributions overlap heavily, confirming the Arctic brightness problem: brightness alone cannot reliably separate the two classes. The not-cloud class shows a notable bimodal structure across radiance angles, reflecting the surface heterogeneity within that category, one peak corresponds to bright, smooth sea ice, and another to darker exposed terrain such as rock or partially melted ice. Clouds, by contrast, form a single unimodal distribution. Notably, the direction of the class difference reverses across angles: at DF (70.5°), clouds are slightly brighter, but at every other angle including nadir, not-cloud pixels are actually brighter on average. No single radiance value reliably indicates cloud presence.

NDAI and SD tell a clearer story. For both features, not-cloud pixels concentrate in a single low-value peak, while cloud pixels shift toward higher values with broader spread. SD shows the most dramatic separation, cloud pixels have a mean value 4.4× higher than not-cloud. CORR is the exception among the engineered features: its distributions for cloud and not-cloud are nearly indistinguishable, consistent with its known limitation that it only reliably detects high-altitude clouds where cross-angle registration shifts are large enough to produce low correlation.

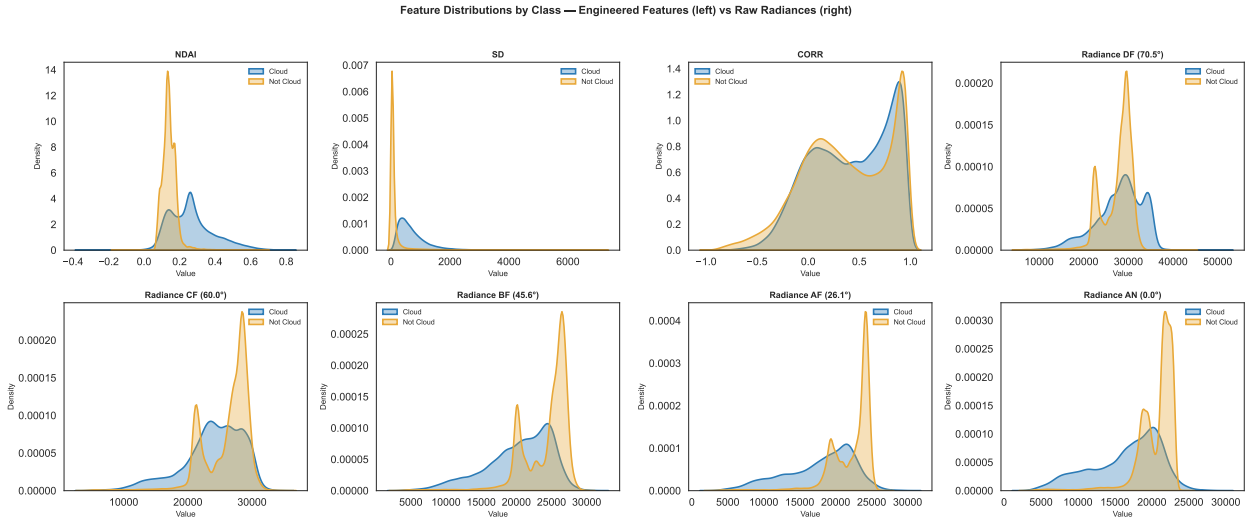

Figure 4: KDE distributions of all eight features by class (cloud in blue, not-cloud in amber), pooled across all three images. Engineered features (left four panels) show substantially clearer class separation than raw radiances (right four panels).

C. Joint Separation via NDAI and CORR

Figure 5 shows the joint density of NDAI and CORR for each class, reproducing the structure of Figure 10 in Shi et al. (2008). The two classes show distinct regions: cloud pixels concentrate at high NDAI and low CORR, while not-cloud pixels dominate the low NDAI and low CORR region. A simple threshold analysis confirms that NDAI is the primary discriminator, 66.6% of cloud pixels have NDAI ≥ 0.20 compared to only 4.7% of not-cloud pixels. CORR provides secondary value within the low-NDAI zone by distinguishing smooth ice (high CORR) from rough terrain (low CORR), but cannot separate low-altitude clouds from the surface since they register similarly across angles.

The overlap region at high CORR and high NDAI (roughly 10% of cloud pixels, 3% of not-cloud) represents the hardest classification cases, likely low-altitude clouds that both features fail to detect, a known limitation of this feature set acknowledged by Shi et al. (2008).

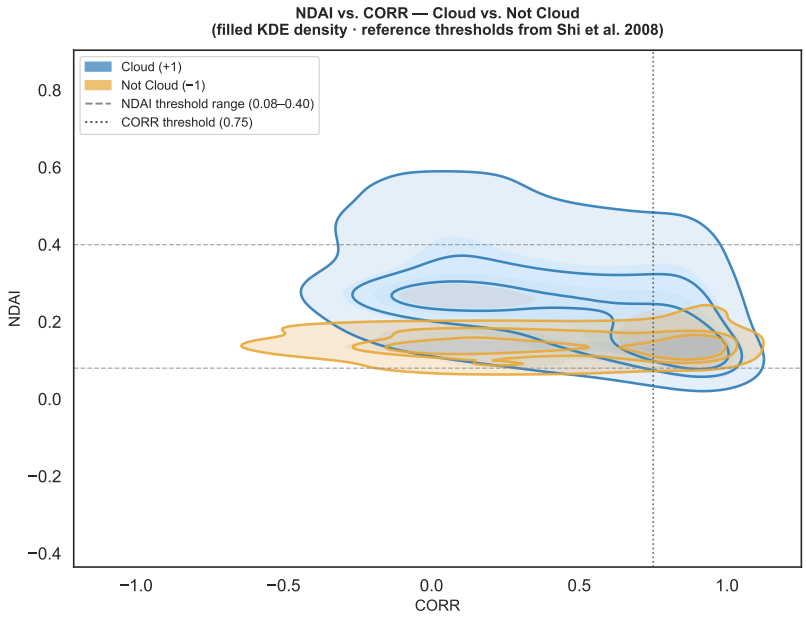

Figure 5: Joint density of NDAI and CORR by class. Dashed lines indicate the threshold ranges from Shi et al. (2008). Cloud pixels concentrate in the high-NDAI, low-CORR region; not-cloud pixels dominate the low-NDAI region regardless of CORR. The overlap at high CORR and high NDAI corresponds to low-altitude clouds that both features struggle to detect.

2.4 Train/Test Split

With only three labeled images, the choice of split was challenging. A random pixel-level split would be inappropriate here: given the strong spatial correlation observed previously, where neighboring pixels almost always share the same label, randomly assigning pixels to train and test would artificially inflate performance by leaking spatial structure across the boundary.

Instead, we adopt a temporal holdout strategy: we train on O012791 and O013257 (spring) and designate O013490 (early summer) as the held-out test set. This mirrors realistic deployment conditions, where a cloud detection algorithm is trained on historically labeled passes and applied to future ones it has never seen. We also included a 5-fold stratified CV on training data to preserve class proportions in each fold.

This design comes with two limitations. First, with only three labeled images available, the entire test set comes from a single satellite pass. This means our performance estimate reflects one specific atmospheric condition on one specific day, so it is hard to know how well the number generalizes. Second, the training set covers a narrower seasonal window than the full April–September collection period, meaning the model may underperform in late summer when surface conditions differ, ice has partially melted, the solar elevation angle has changed, and surface brightness has shifted. The class distribution shift between training (33.2% cloud) and test (47.8% cloud) is itself evidence of this seasonal variation, and we account for it by using class-weighted models.

3 Feature Engineering

3.1 Feature Importance

To identify the most informative features, we quantify class separation using three metrics computed across all labeled pixels: (1) **AUC**, the probability that a randomly chosen cloud pixel ranks higher than a randomly chosen not-cloud pixel; (2) the **KS statistic**, the maximum distance between the two class CDFs; and (3) **Mutual Information (MI)**, which captures any association between a feature and the label, including non-linear relationships.

Rank	Feature	AUC	KS Stat	MI (bits)
1	SD	0.935	0.821	0.428
2	NDAI	0.823	0.621	0.259
3	AF	0.799	0.466	0.202
4	AN	0.794	0.452	0.196
5	BF	0.770	0.445	0.170
6	CF	0.643	0.285	0.087
7	DF	0.554	0.237	0.124
8	CORR	0.524	0.058	0.010

Table 2: Feature importance ranking across all labeled pixels ($n \approx 207{,}000$).

SD ranks first across all three metrics (AUC = 0.935, KS = 0.821, MI = 0.428). Cloud pixels have a mean SD roughly 4.4× higher than not-cloud pixels, reflecting the textural contrast between rough cloud surfaces and smooth Arctic ice.

NDAI ranks second (AUC = 0.823). It measures how much brighter the scene looks from the most extreme forward angle compared to straight down, a ratio that is large for clouds, which scatter light strongly forward, and near zero for flat ice surfaces. Together, SD and NDAI are by far the two strongest individual predictors in the dataset.

CORR ranks last (AUC = 0.524), barely better than a coin flip as a standalone feature. This is surprising given that Shi et al. (2008) rely on it as a key component of their labeling rule, but the numbers are clear: NDAI alone achieves 84% accuracy, while adding CORR's threshold rule on top does not improve it. The likely explanation is that CORR was designed to detect high-altitude clouds, where different camera angles photograph slightly offset parts of the cloud, but the labeled images appear to be dominated by low-altitude clouds, which look the same from all angles and fool CORR into thinking they are clear surfaces. We retain CORR in the feature set anyway, as a learned model may still find it useful in combination with other features even if it contributes little on its own.

Based on this analysis, we identify **SD**, **NDAI**, and **AF** as the three most informative features. SD and NDAI are grounded in both domain knowledge and metrics. AF (AUC = 0.799) emerges as the strongest raw radiance angle, outperforming CORR across all three metrics and providing complementary brightness information not captured by the angular ratio features.

3.2 Feature Engineering

NDAI Variants

The original NDAI compares the most extreme forward camera (DF, 70.5°) to nadir (AN, 0°). We extended this idea to all angle pairs, computing normalized ratios of the form $(a - b)/(a + b)$ for five additional combinations.

Evaluated against the same AUC/KS/MI metrics, **NDAI-DF-AF**, comparing DF (70.5°) to AF (26.1°) instead of nadir, outperforms the original NDAI (AUC 0.833 vs 0.823). We attribute this to the nadir camera (AN) introducing reflectance noise at 0° for smooth ice surfaces, while AF at 26.1° provides a cleaner angular reference. At the other extreme, **NDAI-AF-AN** (AUC $\approx$ 0.500) is effectively useless, confirming that wide angular separation is required for the ratio to carry classification signal. All remaining ratio variants are highly correlated with NDAI-DF-AF ($r > 0.89$) and offer no independent information, so only NDAI-DF-AF is retained.

Feature	Formula	AUC	KS	MI (bits)
NDAI (original)	$(DF - AN)/(DF + AN)$	0.823	0.621	0.259
NDAI_DF_AF	$(DF - AF)/(DF + AF)$	**0.833**	**0.645**	**0.284**
NDAI_CF_AF	$(CF - AF)/(CF + AF)$	0.777	0.545	0.226
NDAI_CF_AN	$(CF - AN)/(CF + AN)$	0.761	0.495	0.186
NDAI_BF_AN	$(BF - AN)/(BF + AN)$	0.666	0.265	0.100
NDAI_AF_AN	$(AF - AN)/(AF + AN)$	0.500	0.122	0.081

Table 3: Comparison of NDAI ratio variants across three separability metrics. NDAI_DF_AF outperforms the original NDAI on all three metrics; NDAI_AF_AN is no better than chance.

PCA on Radiance Angles

The five raw radiances are highly multicollinear ($r = 0.60$–0.97). We applied PCA to the radiance block (fit on training data only, then applied to test) to decompose this redundancy. **PC1** captures 92% of the variance and represents overall brightness, essentially a weighted average across all five cameras. **PC2** captures 6.5% and closely mirrors NDAI: its loadings increase monotonically from AN (-0.511) to DF ($+0.727$), recovering the angular gradient from data rather than from domain knowledge. Despite this, PC2 (AUC = 0.787) is weaker than NDAI-DF-AF (AUC = 0.833) because the linear combination in PC2 is pulled toward the noisy nadir camera. We retain **PC1** as it captures overall brightness orthogonally to the ratio features; PC2 is dropped as it is superseded by NDAI-DF-AF.

Final Feature Set

The final feature set consists of 9 features: SD, CORR, the five raw radiance angles (DF, CF, BF, AF, AN), NDAI-DF-AF, and PC1. We drop the original NDAI in favor of NDAI-DF-AF, which is both stronger empirically and backed by domain knowledge. Raw radiances are retained alongside PC1 because tree-based models can discover ratio-like splits directly from individual angle values without suffering from multicollinearity, unlike linear models. The autoencoder embeddings add a further 32 spatial context features, bringing the total to 41 features passed to the Modeling section.

3.3 Transfer Learning

A major constraint in this problem is the limited amount of labeled data (only three images), compared to a much larger pool of unlabeled images (161 images). This naturally motivates a transfer learning approach: we first learn a general representation of the data using all available images, and then reuse this representation for downstream classification.

Motivation

From the EDA, it is clear that cloud pixels are not independent. They form spatially coherent regions with distinct textures. However, all handcrafted features (e.g., NDAI, SD) are defined at the single-pixel level and ignore this spatial context.

Learning spatial features directly from the labeled data is not feasible given the extremely limited sample size. Instead, we exploit the large set of unlabeled images and use an autoencoder to learn representations of local patches in an unsupervised way. This allows the model to capture patterns such as:

- cloud texture (irregular, high-frequency variation),
- smooth ice or snow surfaces,
- transitions at cloud boundaries.

This design choice reflects a key trade-off: rather than relying purely on physics-inspired features, we allow the model to learn data-driven spatial structure that is otherwise difficult to hand-engineer.

Design Choices and Practical Constraints

In principle, we could extract all possible patches from all images, but this would result in tens of millions of samples (on the order of tens of GB in memory), which is not practical even on most HPC setups.

To address this, we randomly sample a fixed number of patches per image. We set this to 5000 patches per

image, which was a practical compromise:

- large enough to capture diverse spatial patterns within each image,
- small enough to keep memory usage under ∼1GB,
- fast enough to allow multiple experimental runs.

We experimented with smaller values (e.g., 2000), which were faster but produced slightly noisier embeddings (less stable across runs). Increasing beyond 5000 provided only marginal improvements while significantly increasing training time and memory overhead, so we settled on this value.

From an implementation perspective, this sampling strategy was also important to avoid memory bottlenecks when constructing the dataset in PyTorch. Without subsampling, the patch extraction step alone becomes a major computational bottleneck.

For efficiency, we used a two-stage workflow: local runs (with fewer patches and epochs) were used for debugging and architecture tuning, while the final training was conducted on an HPC environment with GPU acceleration. This allowed us to iterate quickly during development while still training a stable final model.

Autoencoder Architecture

We train the autoencoder on 9×9 patches centered at each pixel. Each patch contains 8 channels (engineered features and radiances), resulting in an input dimension of $8 \times 9 \times 9 = 648$.

The model consists of an encoder that maps the patch into a low-dimensional embedding, and a decoder that reconstructs the original patch from this embedding.

We modified the baseline architecture provided in the starter code in several ways:

- **Increased embedding dimension (8 → 32):** The original bottleneck compresses the input by a factor of ∼81×, which we found too aggressive. In practice, this led to underfitting (high reconstruction error and overly smooth outputs). Increasing the embedding size to 32 significantly improved reconstruction quality while still enforcing meaningful compression.
- **Convolutional structure:** Instead of fully flattening the input, we replaced parts of the encoder with convolutional layers to better exploit spatial locality. This required minor changes to the data pipeline (reshaping inputs to channel-first format) and yielded more stable training.
- **Training stability tweaks:** We adjusted the learning rate (5×10^{-4}) and added weight decay (10^{-5}) to stabilize training. We also monitored validation loss to implement early stopping, as we observed that training beyond convergence did not improve reconstruction.

The model is trained using mean squared reconstruction loss:

$$\mathcal{L} = \|x - \hat{x}\|^2$$

Training Behavior

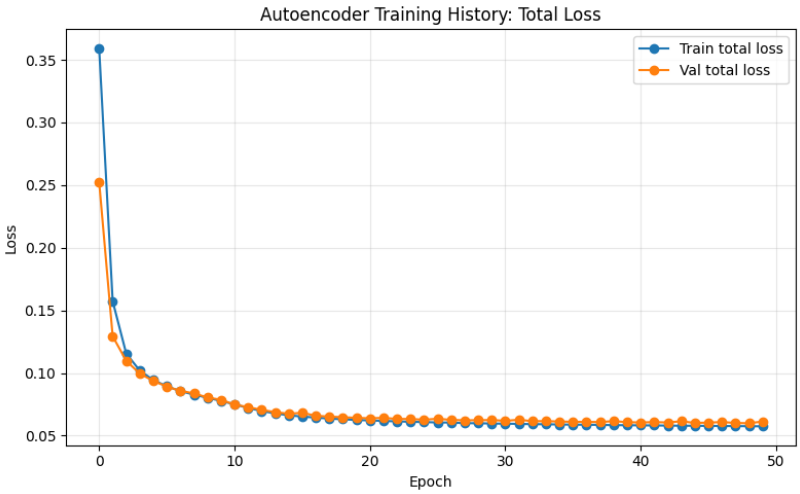

Figure 6: Autoencoder training curve (total loss). The rapid decrease in early epochs followed by a plateau indicates that the model efficiently learns dominant spatial structures and converges without overfitting.

Figure 6 shows the training dynamics of the autoencoder in terms of total loss. We observe three key patterns.

- **Fast initial convergence**: First, there is fast initial convergence. The loss decreases sharply within the first 5–10 epochs. This suggests that the model quickly captures the most salient structures in the data, such as overall brightness patterns and coarse spatial textures that distinguish cloud from non-cloud regions.
- **Stable plateau**: Second, the loss exhibits a stable plateau after approximately 15 epochs. This indicates that the model has largely saturated its representational capacity given the chosen architecture and embedding dimension. Additional training yields minimal improvement, implying that the learned latent representation has converged.
- **Minimal overfitting**: Third, there is minimal overfitting. The training and validation losses closely track each other throughout the training process. This suggests that the learned features generalize well across different images, which is particularly important given the limited number of labeled samples and the reliance on transfer learning.

Based on this behavior, we employ early stopping to prevent unnecessary training beyond convergence and to select the model checkpoint with the best validation performance. Overall, the training dynamics indicate that the autoencoder successfully learns a compact and stable representation of local spatial patterns, which can be leveraged as additional features for downstream classification.

Feature Extraction and Transfer

After training, we discard the decoder and use the encoder to extract a 32-dimensional embedding for every pixel in both labeled and unlabeled images.

From a practical standpoint, we precompute these embeddings and store them as additional features, rather than recomputing them on the fly during model training. This significantly reduces training time for downstream classifiers.

These embeddings capture local spatial structure that is not available in single-pixel features, including:

- texture variability (cloud vs. smooth ice),
- edge structures at cloud boundaries,
- consistency of multi-angle radiances within local neighborhoods.

We append these embeddings to the handcrafted feature set:

$$\text{features} = \{\text{SD, CORR, radiances, NDAI_DF_AF, PC1}\} \cup \{\text{AE}_1, \ldots, \text{AE}_{32}\}$$

This results in a total of 41 features per pixel.

Interpretation

This pipeline can be viewed as transfer learning in the following sense:

- the autoencoder is pre-trained on a large unlabeled dataset to learn general spatial representations,
- these representations are then transferred to the supervised classification task, where labeled data is limited.

Conceptually, the handcrafted features capture physics-based signals (angular scattering and variability), while the autoencoder captures data-driven spatial structure. These two sources of information are complementary, and together they help resolve challenging cases such as low-altitude clouds that resemble ice in radiance values but differ in local texture.

We emphasize that the autoencoder is not optimized for classification performance directly, but rather for reconstructing the input distribution. Its usefulness is evaluated through downstream predictive performance. In Section 4, we evaluate the impact of these embeddings on classification performance.

4 Modeling

4.1 Problem Setup

The goal is to classify cloud presence (+1 for cloud, −1 for not cloud) using Multi-angle Imaging SpectroRadiometer (MISR) radiance data. Our modeling pipeline adopts a temporal holdout strategy: the training set consists of images `O012791` and `O013257`, while the image `O013490` serves as a temporal holdout test set. During training and testing, we only use the labeled pixels, This split reflects the real-world challenge of deploying a model on future satellite orbits. The features used include the original MISR radiance angles (DF, CF, BF, AF, AN), expert-engineered features (NDAI_DF_AF, SD, CORR), and a suite of 32 latent features derived from our autoencoder (AE) model.

4.2 Models Compared

There are three machine learning methods we applied, namely Ensemble Learning (including Random Forest and Boosting), Logistic Regression, and Support Vector Machine. For Ensemble Learning, we evaluate four main configurations to understand the impact of feature engineering and regularization:

1. **Random Forest (Literature):** A baseline RF model using only the following three features: `SD`, `CORR`, and new `NDAI_DF_AF`.
2. **Histogram Gradient Boosting (Base):** An efficient gradient-boosted decision tree implementation utilizing only the 3 literature features and original features(DF, CF, BF, AF, AN).
3. **Random Forest (Full):** A Random Forest ensemble trained on all 40 features (8 base + 32 AE).
4. **Histogram Gradient Boosting (Full):** The HGB architecture trained on the full 40-feature set.

For Logistic Regression and SVM, we have two variations each:

1. **LASSO Logistic Regression:** A Logistic Regression model with Lasso selecting existing features using L1 penalty.
2. **Stepwise Logistic Regression:** Logistic Regression model with features generated by stepwise selection (with accuracy as the scoring criterium).
3. **Tuned SVM:** SVM model trained on 8 selected features with its parameters tuned through Grid Search methodology.
4. **SVM (Base):** SVM trained on the three engineered features applied in the paper: `SD`, `CORR`, and `NDAI`.

4.3 Model Assumptions and Reasonableness

Both Random Forest and Gradient Boosting are tree-based ensemble methods. These models are nonparametric and do not require strong assumptions about the underlying distribution of the data (such as linearity or normality of residuals). However, they are sensitive to covariate shifts between training and testing data. The logistic regression model assumes a linear relationship between the logit transformation of response variables and the predictor variables. SVM assumes a set of support vectors serving as margins that form a hyperplane in a kernel space to separate the binary categories. Following an extensive grid search for parameter tuning, the L1 penalty coefficient for the Lasso logistic regression was set at 0.8 and the penalty parameter for the support vector machine was fixed at 1.0.

We assessed the temporal stability by calculating the Standardized Mean Difference (SMD) and skewness shifts between the training images and the temporal holdout. Significant distribution shifts were observed in several features:

- **NDAI_DF_AF:** $|SMD| = 1.147$; training skew = 2.481, testing skew = 1.226.
- **DF (Radiance angle):** $|SMD| = 0.748$; training skew = -0.676, testing skew = -1.762.
- **CORR:** $|SMD| = 0.537$; training skew = -0.639, testing skew = 0.099.

Tree-based models remain reasonable choices due to their ability to model complex, non-linear interactions without requiring specific parametric forms, though they must be monitored for performance degradation in new environments, while both Logistic Regression and SVM are intuitively suitable for this research because the fundamental nature of the task is binary classification.

4.4 Fit Assessment

Model performance was evaluated using the temporal holdout test (Table 4). All tree models achieved high ROC AUC values (> 0.99). Interestingly, the literature-based RF performs remarkably well, suggesting the three core features carry the vast majority of the signal. The models with full features provide a slight edge in Accuracy.

Model	Accuracy	ROC AUC	F1 Score	Precision	Recall
RF (Literature)	0.9559	0.9945	0.9558	0.9170	0.9981
HGB (Base)	0.9608	0.9958	0.9605	0.9283	0.9950
RF (Full)	0.9732	0.9941	0.9725	0.9564	0.9892
HGB (Full)	0.9787	0.9976	0.9780	0.9660	0.9903
Lasso (Full)	0.4973	0.5109	0.4957	0.5148	0.5065
StepWise (Full)	0.8200	0.8101	0.8113	0.8274	0.8148
SVM (Base)	0.9563	0.9487	0.9523	0.9465	0.9435
SVM (Full)	0.4202	0.4148	0.3569	0.5152	0.4878

Table 4: Performance metrics on the temporal holdout set (`0013490`). HGB (Base) is selected as the primary model.

4.5 Selected Model Diagnostics (HGB-Base)

The ROC curve analysis of out selected model (Figure 7) shows that while all models perform well, zooming into the top-left corner reveals that HGB-Base and HGB-Full maintain a slightly more "square" curve than the RF variants, like Stepwise Logistic Regression (Figure 8).

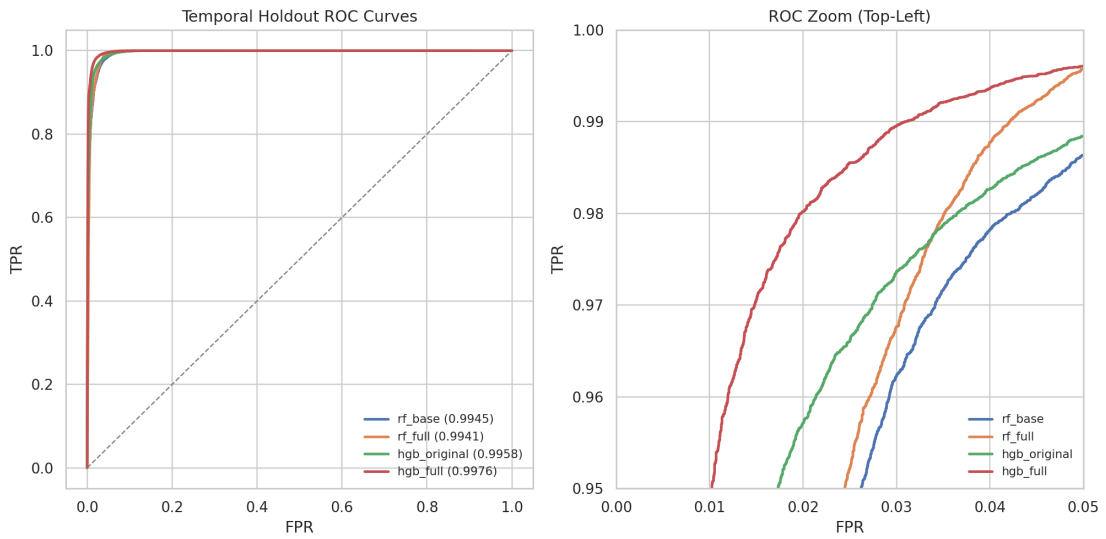

Figure 7: Temporal Holdout ROC Curves with zoom. All models are highly competitive, with HGB variants showing slight dominance in the high-recall/high-precision regime.

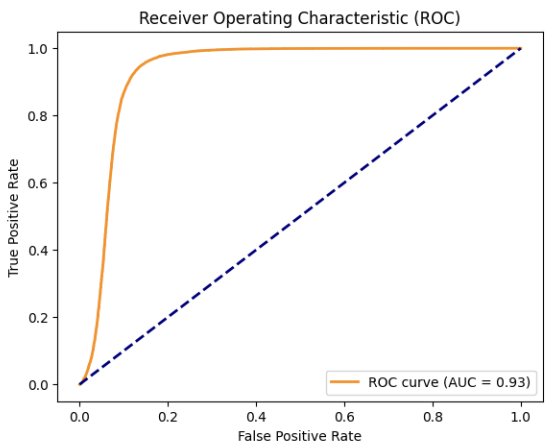

Figure 8: ROC Curve from Logistic Regression.

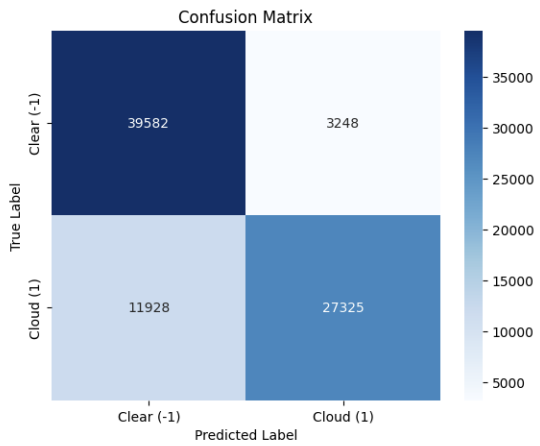

Figure 9: Confusion Matrix from Logistic Regression.

The permutation feature importance analysis (Figure 10) reveals that `SD` is the most dominant feature. The confusion matrix (Figure 11) shows that the model is particularly strong at identifying clouds (Recall = 0.9950).

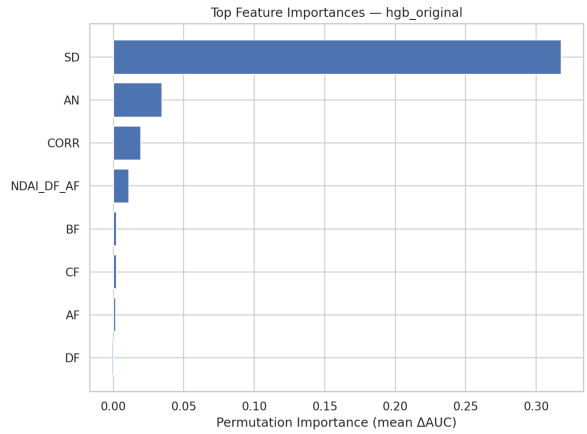

Figure 10: Top features by permutation importance for the HGB model.

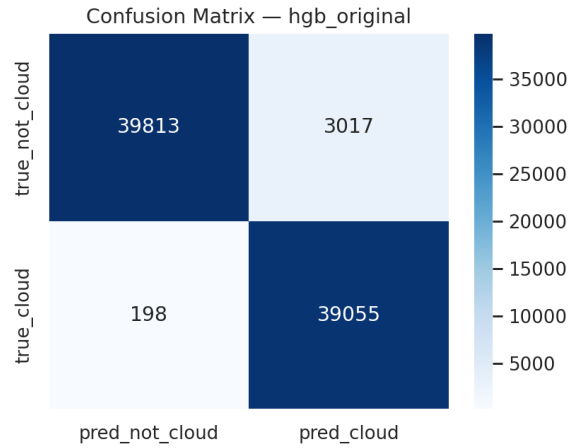

Figure 11: Confusion matrix for HGB on the temporal holdout set.

4.6 Post-hoc Error EDA and Boundary Analysis

Spatial visualization of misclassifications (Figure 12) reveals that errors cluster at the boundaries of clouds. To quantify this effect, we calculated the Euclidean distance from each pixel to the nearest pixel of the opposite class (the "label boundary").

While the theoretical minimum distance between adjacent grid pixels is 1.0 unit, our analysis of the temporal holdout set (`O013490`) revealed an empirical minimum distance of **6.0** units between labeled pixels of opposite classes. This indicates a spatial buffer or a lower-resolution sampling in the expert-provided labels for this orbit.

Despite this labeling gap, the quantitative trend (Figure 13) is clear: the error rate spikes to $\approx 50\%$ at this observable 6-unit threshold and decays rapidly to $< 1\%$ for pixels located more than 100 units from a transition. This confirms that the model's uncertainty is almost entirely localized at spatial boundaries.

Presumably out of the same reason, similar location patterns appear in spatial distribution of misclassified data points in SVM model (Figure 14). With a lower accuracy, Logistic Regression Model is not able to capture a piece of cloud in the middle of the graph, reflecting lack of further information retrieval from selected features (Figure 15).

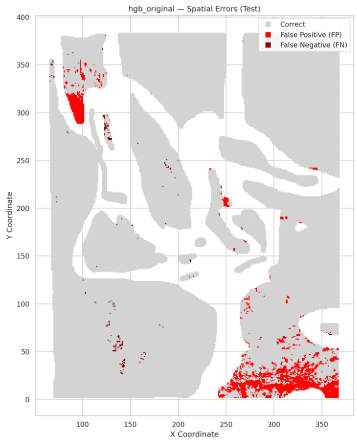

Figure 12: Spatial map of classification errors concentrated at cloud edges.

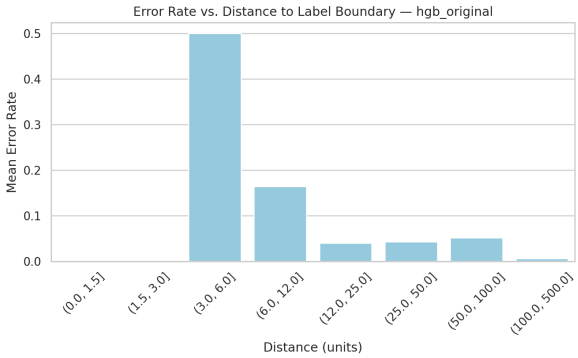

Figure 13: Mean Error Rate vs. Distance to Label Boundary.

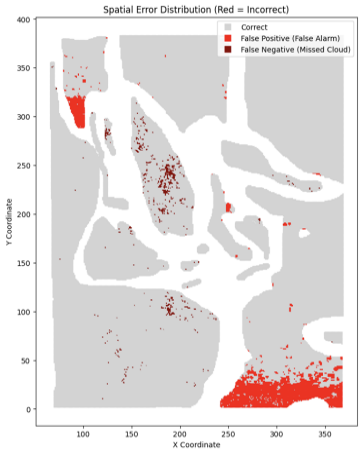

Figure 14: Spatial map of classification errors in the SVM (Lit) model setting.

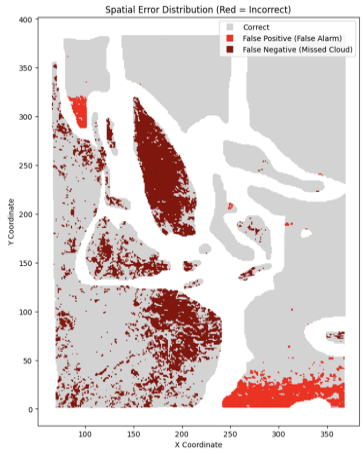

Figure 15: Spatial map of classification errors in the Stepwise Logistic Regression model.

Additionally, the diagnostic assessment of the SVM model was conducted by employing PCA to visualize both the decision clusters and the support vector density in a reduced-dimensional space (Figure 16). The results of this projection demonstrate a high degree of internal consistency. Specifically, a visible double peak emerges within the two-dimensional principal component space, characterized by support vectors distributed strategically along the decision boundaries to separate the two binary classes. This spatial arrangement confirms that the model has identified a stable structural manifold for classification.

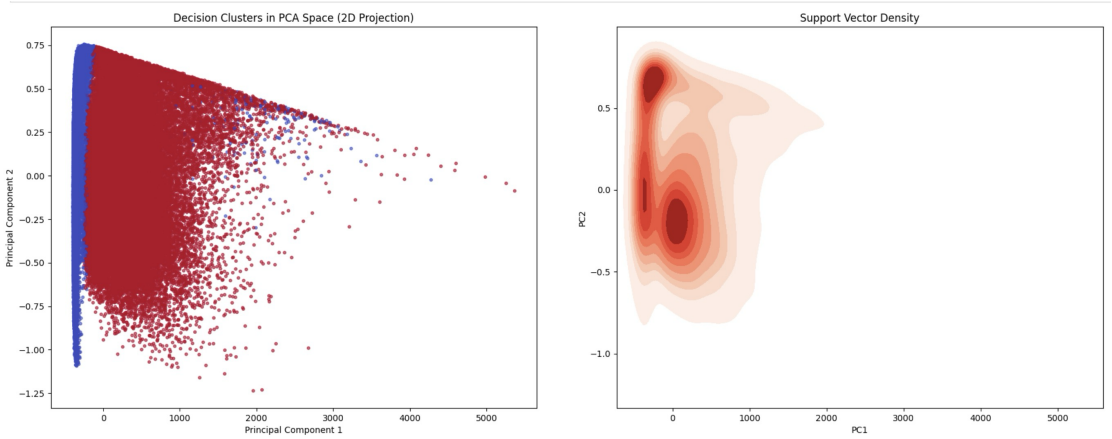

Figure 16: PCA visualizations of both the decision clusters and the support vector density into 2 dimensions. Visible trend of double peaks are coherent with the robust performance of the SVM model.

For non-ensemble learning models, the primary change would be collinearity and overfitting. Lasso Logistic Regression and SVM seem to be severely influenced by the difference of features included. With SVM, the best model included only three features in the model `SD`, `CORR`, and `NDAI`. As seen from model metrics discussed above (Figure 4), adding one additional feature into the predictors (the first `PC`, for example) will immediately compromise the accuracy score on the test set down to approximately 0.4. In other words, even with non-linear kernel function, additional dimensionality in predictors will bring radical change to the performance of SVM. Similarly, the features selected by Lasso regression include 7 variables, which has 2 more compared to the best performance given by stepwise regression, the corresponding collinearity problem already reflected in the fit assessment. By constraining the complexity of the feature space and regularizing the optimization objectives, the models are better equipped to generalize beyond the training data.

Investigation into the features driving these boundary errors revealed a significant shift in `CORR` (correlation). The mean `CORR` for pixels near boundaries (< 15 units) is **0.473**, compared to only **0.097** for pixels deep within homogeneous regions. This suggests that the high-frequency textures associated with cloud edges or fragmented cover are the primary challenge for the ensemble learning model, as they present feature signatures that are intermediate between clear sky and dense cloud.

We further analyzed the distributions of key features across four groups: True Negatives (TN), True Positives (TP), False Positives (FP), and False Negatives (FN). The statistical summaries (Table 5) and distributions (Figure 17) reveal distinct feature signatures for misclassifications.

Feature	Metric	TN	TP	FP	FN
`SD`	Mean	89.82	705.98	773.81	**1376.21**
	Median	37.39	619.14	622.54	**1032.57**
`CORR`	Mean	0.263	0.196	**0.823**	0.667
	Median	0.230	0.161	**0.909**	0.791
`NDAI`	Mean	0.109	0.299	0.177	0.235
	Median	0.103	0.262	0.182	0.205

Table 5: Feature distribution statistics by classification group. High `SD` drives missed clouds (FN), while high `CORR` is the primary driver of false clouds (FP).

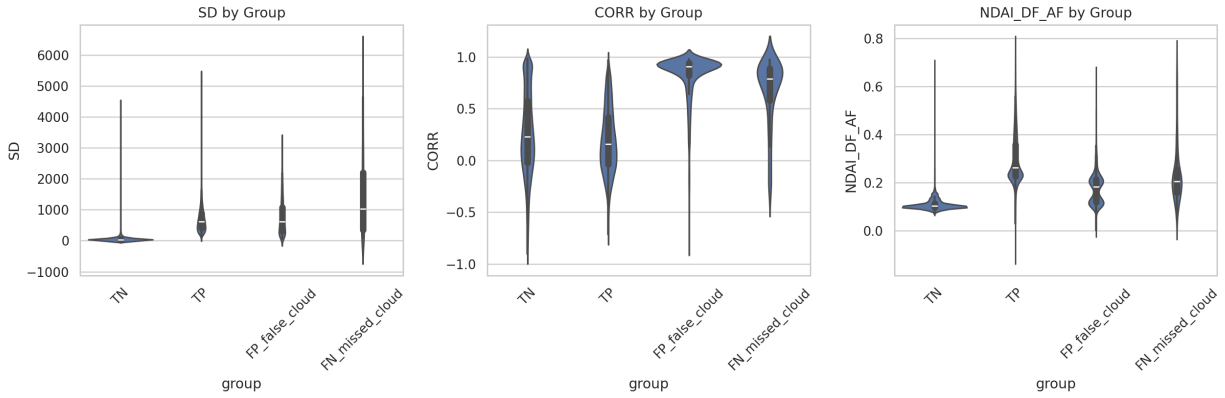

Figure 17: Violin plots of feature distributions across classification groups. Missed clouds (FN) are characterized by extreme spatial deviation (SD), whereas false clouds (FP) show exceptionally high correlation (CORR).

The analysis indicates two primary failure modes:

1. **Texture-Driven False Negatives:** The missed clouds (FN) exhibit substantially higher `SD` values (mean 1376.2) than the correctly identified clouds. This suggests that the model fails to detect clouds in regions of extreme heterogeneity or high-frequency surface features, where the cloud signal is "masked" by spatial complexity.
2. **Correlation-Driven False Positives:** False clouds (FP) are strongly associated with extremely high `CORR` values (median 0.909). These high correlation signatures likely arise from smooth clear-sky surfaces (e.g., ice or water at specific angles) that mimic the smoothness typically associated with cloud layers, leading the model to overestimate cloud probability.

4.7 Stability and Future-Data Readiness

4.7.1 Predictions on Unlabeled Pixels

Expert labelers assigned `label` = 0 to pixels that were ambiguous at the time of annotation. For the 32,949 unlabeled pixels from orbit `0013490`, we applied the trained RF to these pixels to probe the model's behavior in the ambiguous regime.

The model classifies **82.7%** (27,244) of unlabeled pixels as cloud and only 17.3% (5,705) as non-cloud (Figure 18). The probability histogram (Figure 21) shows a strongly bimodal distribution: most unlabeled pixels receive near-zero or near-one cloud probabilities, with relatively few landing in the $[0.3, 0.7]$ uncertain band.

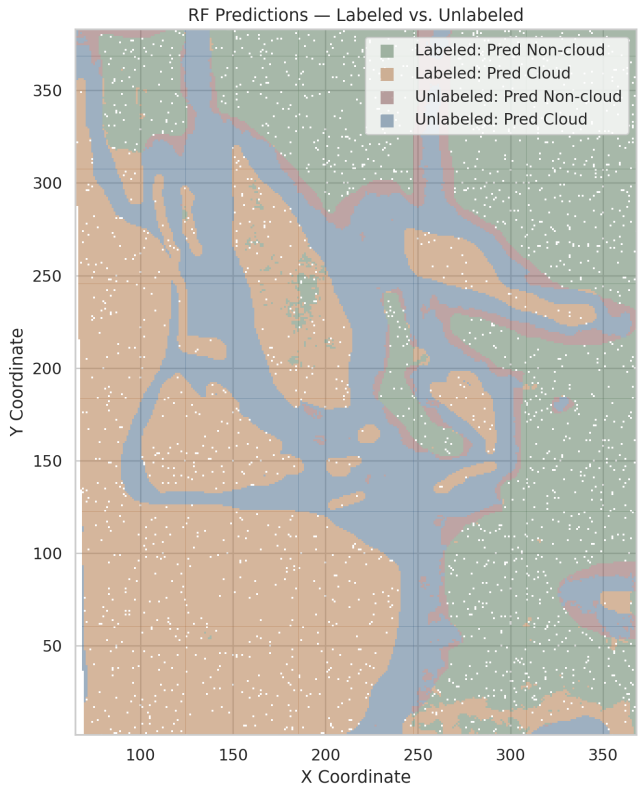

Figure 18: Spatial map of orbit `0013490` showing RF predictions for all pixel types.

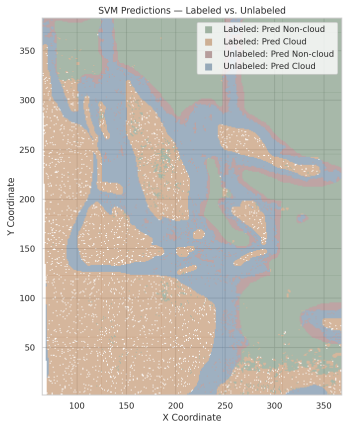

Figure 19: SVM spatial map

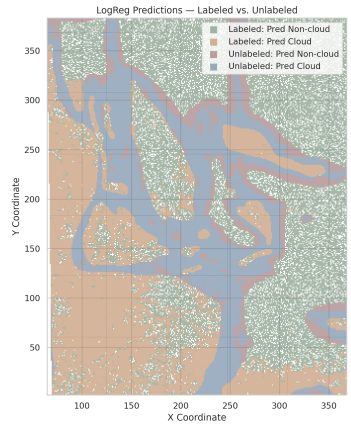

Figure 20: Stepwise Logistic Regression spatial map

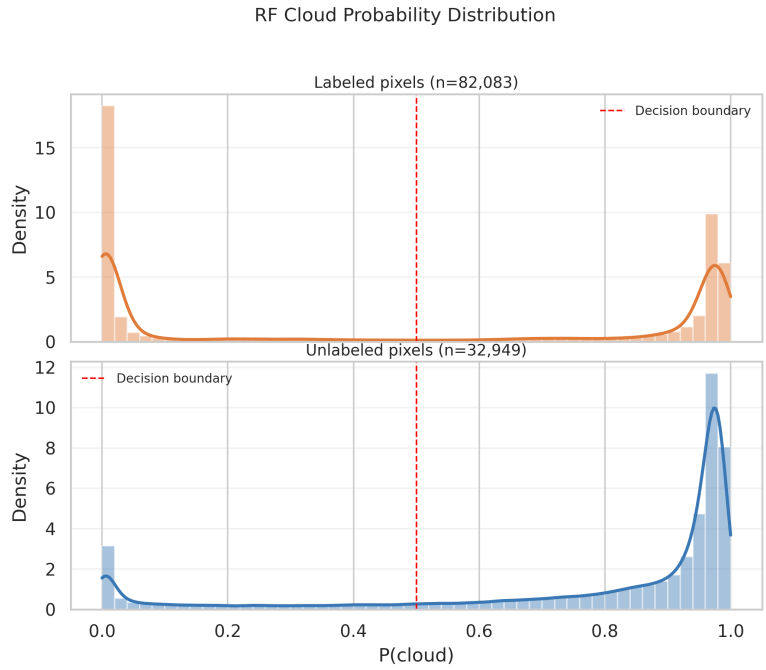

Figure 21: RF cloud probability distributions for labeled (top) and unlabeled (bottom) pixels in orbit `0013490`, shown in separate panels with a shared x-axis. Smooth curves are Gaussian KDE estimates ($h = 0.05\hat{\sigma}$, narrow bandwidth chosen to preserve the sharp bimodal peaks).

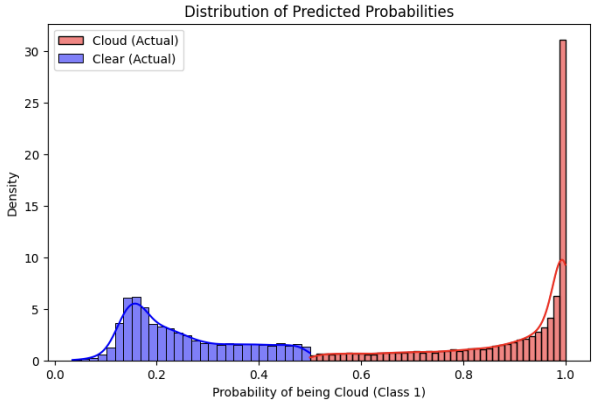

Figure 22: Logistic Regression cloud probability distributions for labeled and unlabeled pixels. Massing trends appear to be similar to RF results.

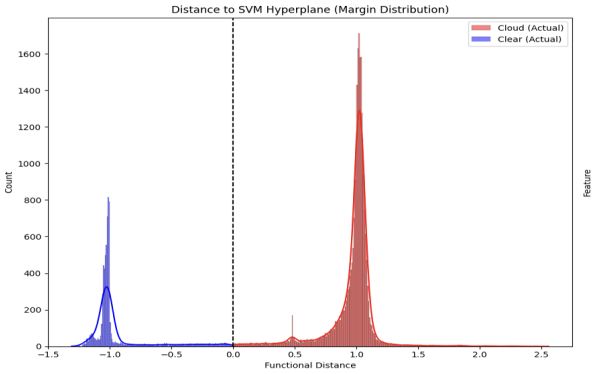

Figure 23: Distance from the unlabeled pixels to the dividing hyperplane in SVM model. Pattern of double peaks is also visible.

The strong cloud-skew (83%) is consistent with where unlabeled pixels spatially appear: expert annotators tend to leave ambiguous labels at cloud boundaries and in partially-obscured regions, which are surrounded by labeled cloud pixels. The RF, drawing on spatial texture features (especially `SD`) and AE embeddings that encode local patch structure, reads these as cloud-like.

4.7.2 Label Noise Robustness

To assess sensitivity to annotation errors in the training set, we randomly flipped 5% of training labels ($y \to -y$ with probability 0.05) and retrained the RF from scratch using the same hyperparameters. This was repeated 3 times with independent flip draws (≈6,250 of 125,598 labels corrupted per run).

Condition	ROC AUC	F1	Accuracy
Clean labels	0.9941	0.9712	0.9720
5% flipped (mean)	0.9931	0.9725	0.9732
5% flipped (std)	0.0003	0.0009	0.0009
Δ (flipped − clean)	−0.0010	+0.0013	+0.0012

Table 6: RF test-set performance under 5% training label flip, averaged over 3 independent repetitions. All metrics are evaluated on the clean temporal holdout (`0013490`).

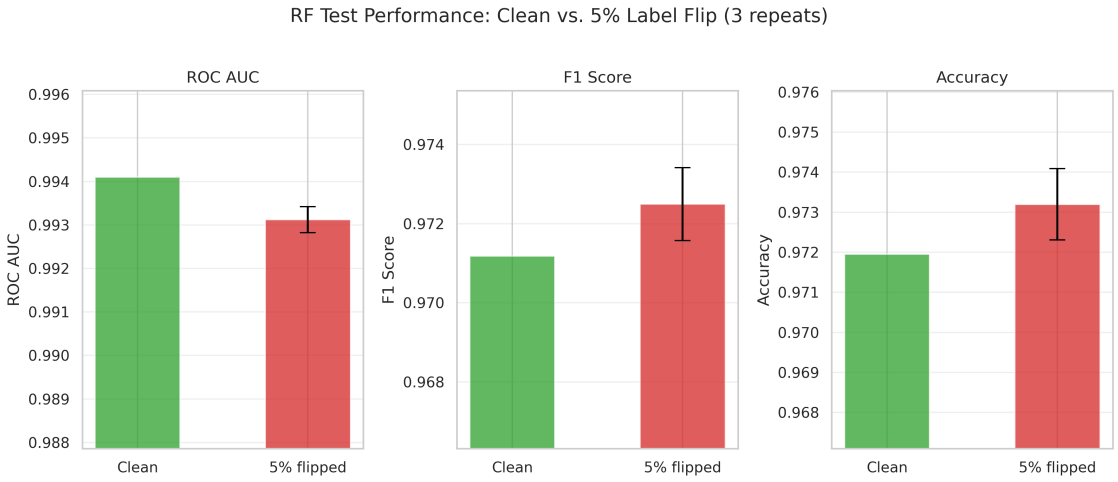

Figure 24: Bar chart comparing RF test metrics between clean and 5%-flipped training labels. Error bars show standard deviation across 3 independent flip draws. The y-axis is zoomed to the relevant range to make the (small) differences visible.

This robustness is expected from a Random Forest: the bagging mechanism means each tree is trained on a bootstrap sample, so the corrupted labels are spread across trees and their influence is diluted by the ensemble vote. In practice, this implies the model would remain reliable even if expert annotators disagreed on roughly 1 in 20 pixels. In other models other than ensemble learning, noise does compromises model performance to a certain extent. Logistic Regressions have all three metrics lowered after the flipping process. SVM exhibits a strong robustness against added noise. All three metrics have extremely little change compared to the original training result.

5 Bibliography

References

[1] Shi, T., Yu, B., Clothiaux, E. E., and Braverman, A. J. (2008). Daytime Arctic cloud detection based on multi-angle satellite data with case studies. *Journal of the American Statistical Association*, 103(482):584–597.

A Academic honesty

A.1 Statement

The work in this report is our own and all the sources used are properly cited. Academic research honesty is important because science is built on truth.

A.2 LLM Usage

Coding

- Team member 1: I used Anthropic's Claude to understand better how to improve the autoencoder to make it work with less RAM, to suggest ways to "prettify" some of the visuals from EDA (see figure 4 and 5) and for general documentation.
- Team member 2: I used Gemini to brainstorm possible ways of implementing SVM and Regression models. I also used this model to polish my already-existing writings.
- Team member 3: I used ChatGPT to clean the code, create documentation, and brainstorm the method in order to improve the performance of the autoencoder based on image data.
- Team member 4: I used Claude Code to brainstorm ideas, create documentation, and help me plot figures.

Writing

- Used to find grammatical errors and LaTeX formatting (e.g. tables).